

Evaluating Ray as a Unified Distributed Compute Framework for Large-Scale Healthcare Analytics: An End-to-End Case Study on MIMIC-IV

Tran Tuan Anh, Tran Dang Duat, Nguyen Anh Khang, Nguyen Trung Kien, Nguyen Van Vuong
VNU University of Engineering and Technology
Hanoi, Vietnam

Tóm tắt nội dung—Bài báo cáo đánh giá Ray như một nền tảng tính toán phân tán thống nhất cho bài toán phân tích y tế quy mô lớn trong hệ thống PREDICTCARE AI. Bối cảnh nghiên cứu xuất phát từ nhu cầu kết hợp dữ liệu chuỗi thời gian sinh hiệu với hồ sơ nhân khẩu học và nhập viện trong MIMIC-IV để dự báo rủi ro sinh tồn cho bệnh nhân. Các pipeline truyền thống thường bị phân mảnh giữa tầng ETL và tầng học máy, dẫn đến chi phí ghi/đọc đĩa lớn, nguy cơ lỗi tràn bộ nhớ RAM hệ thống trong quá trình kết nối và nén dữ liệu khổng lồ, cũng như sự chậm trễ khi chuyển giao dữ liệu sang môi trường huấn luyện. Để giải quyết vấn đề đó, nghiên cứu thiết kế một pipeline toàn trình dựa trên Ray trên cụm phân tán không đồng nhất gồm head node thiên về CPU, HDFS làm tầng lưu trữ, và edge node sở hữu GPU RTX 3060 12GB. Ray Data được dùng để thực hiện distributed join giữa các bảng *patients*, *admissions* và *chartevents*, sau đó tạo tập Gold Data phục vụ mô hình XGBoost Survival Analysis. Ray Train tiếp nhận trực tiếp tập dữ liệu đặc tả đã được nén mạnh từ phase trước để thực hiện huấn luyện trong bộ nhớ GPU. Kết quả cho thấy cách tiếp cận thống nhất này giúp giảm bottleneck do disk I/O, giải quyết bài toán kết nối dữ liệu phân tán nhiều bảng với mức tiêu thụ RAM được kiểm soát, và tận dụng GPU chủ yếu cho tăng tốc hội tụ của mô hình thay vì để giải quyết bài toán dung lượng dữ liệu thô.

Index Terms—Ray, Ray Data, Ray Train, MIMIC-IV, Survival Analysis, Distributed Join, Clinical Decision Support System.

I. INTRODUCTION

A. Bối cảnh nghiên cứu

Sự kết hợp giữa Big Data và Trí tuệ nhân tạo đang tái định hình các hệ thống hỗ trợ quyết định lâm sàng (Clinical Decision Support System - CDSS), đặc biệt trong các bài toán cần tích hợp dữ liệu đa luồng để dự báo nguy cơ cho bệnh nhân. Trong bối cảnh đó, bộ dữ liệu MIMIC-IV trở thành chuẩn tham chiếu quan trọng cho nghiên cứu y sinh tính toán nhờ chứa đồng thời thông tin nhân khẩu học, lịch sử nhập viện và các chuỗi thời gian sinh hiệu dày đặc. Đối với PREDICTCARE AI, giá trị cốt lõi không nằm ở việc phân tích một bảng riêng lẻ, mà ở khả năng hợp nhất dữ liệu từ nhiều bảng để tạo ra các đặc trưng lâm sàng có ý nghĩa cho dự báo sinh tồn[cite: 1].

Cụ thể, ba bảng *patients*, *admissions* và *chartevents* đóng vai trò trung tâm. Bảng *patients* cung cấp thông tin nhân khẩu học và trạng thái tử vong; bảng *admissions* xác định bối cảnh nhập viện và khung thời gian diễn ra biến cố; còn bảng *chartevents* chứa hàng trăm triệu bản ghi đo đặc sinh hiệu theo thời gian. Việc kết nối đúng ba nguồn này là điều kiện tiên quyết để

xây dựng một pipeline dự báo nguy cơ tử vong có giá trị ứng dụng thực tế.

a) TO DO:

- Thu thập 2–3 tài liệu nền về vai trò của Big Data, time-series và AI trong CDSS để mở đầu phần này bằng phát biểu có căn cứ.
- Bổ sung số liệu mô tả ngắn về MIMIC-IV và quy mô của bảng *chartevents* để làm rõ vì sao đây là benchmark đủ nặng cho bài toán phân tán.
- Viết thêm 2–3 câu nối giữa mục tiêu học thuật và nhu cầu thực tế của PREDICTCARE AI, tránh để phần mở bài chỉ mang tính mô tả dataset.
- Chèn citation tương ứng cho nhận định về tầm quan trọng của dữ liệu đa bảng trong phân tích lâm sàng.

B. Nỗi đau công nghệ của pipeline truyền thống

Mặc dù dữ liệu y tế ngày càng phong phú, pipeline xử lý trong thực tế vẫn thường bị chia cắt giữa nhiều công cụ: một hệ thống cho ETL phân tán, một hệ thống khác cho huấn luyện mô hình, và thêm các bước trung gian để tuần tự hóa, lưu file và nạp lại dữ liệu. Kiến trúc này tạo ra ba nút thắt lớn. Thứ nhất là *disk I/O bottleneck*, khi dữ liệu sau các bước join, aggregation và feature engineering phải ghi xuống đĩa hoặc data lake trước khi được đọc lại cho pha huấn luyện. Thứ hai là độ phức tạp vận hành do phải phối hợp nhiều runtime khác nhau, làm tăng chi phí cấu hình và khó khăn trong giám sát end-to-end. Thứ ba là rủi ro lỗi *out-of-memory* ở RAM hệ thống trong quá trình kết nối và nén các bảng lâm sàng khổng lồ, đặc biệt khi *chartevents* phải được join và aggregate cùng với các bảng ngữ cảnh như *patients* và *admissions*.

Trong các kiến trúc cũ kiểu PySpark cộng với một framework ML tách biệt, bài toán không chỉ là xử lý chậm mà còn là chuyển giao dữ liệu rườm rà giữa các pha. Đặc biệt với pipeline đa bảng, các bước distributed join giữa bảng lớn và bảng nhỏ, sau đó tiếp tục nối sang feature table time-series, thường phát sinh overhead đáng kể về shuffle, serialization và materialization trung gian. Vì vậy, nút thắt không chỉ nằm ở một phép toán đơn lẻ, mà nằm ở toàn bộ quy trình từ raw data đến model.

a) TO DO:

- Liệt kê rõ “hệ thống cũ” của nhóm đang gồm những thành phần nào: PySpark, HDFS, script feature engineering, bước export file trung gian, bước train XGBoost.

- Thu thập 1–2 ví dụ thực tế về bottleneck: thời gian ghi parquet, thời gian reload dữ liệu cho training, hoặc log System OOM khi join/aggregate dữ liệu lớn ở head node.
- Viết thêm một đoạn so sánh định tính giữa pipeline phân mảnh và pipeline unified để tạo tiền đề hợp lý cho việc chọn Ray.
- Nếu có thể, bổ sung một bảng nhỏ “pain point → impact” để tăng sức thuyết phục ở phần mở đầu.

C. Bài toán thực tiễn của PREDICTCARE AI

Nghiên cứu này xuất phát từ nhu cầu triển khai một pipeline toàn trình cho PREDICTCARE AI, trong đó khoảng 300 triệu bản ghi sinh hiệu từ bảng *chartevents* cần được kết nối với thông tin từ *patients* và *admissions* để tạo dữ liệu huấn luyện cho mô hình sinh tồn. Bài toán hệ thống được chia thành ba lớp: (1) xác định nhân sinh tồn từ dữ liệu tử vong và thời gian nhập viện; (2) nén khối lượng dữ liệu chuỗi thời gian khổng lồ thông qua các phép windowing và aggregation trong 24 giờ hoặc 48 giờ đầu; và (3) kết hợp nhân với đặc trưng để tạo ra Gold Data hoàn chỉnh phục vụ huấn luyện GPU.

Kiến trúc cũ dựa trên cụm CPU truyền thống tuy giải quyết được bài toán lưu trữ, nhưng chưa xử lý tốt vấn đề chuyển giao dữ liệu giữa ETL và Machine Learning[cite: 1]. Do đó, hệ thống cần một nền tảng thống nhất có thể vừa đóng vai trò data engine cho pipeline đa bảng, vừa điều phối tài nguyên huấn luyện phân tán.

a) TO DO:

- Viết rõ hơn workflow hiện tại của PREDICTCARE AI: raw data đi vào đâu, ETL xong ra bảng gì, model hiện đang train trên dữ liệu nào.
- Mô tả cụ thể ba lớp bài toán bằng ngôn ngữ hệ thống: input, processing step, output của từng lớp.
- Bổ sung sơ đồ mini hoặc bullet sequence từ *patients/admissions/chartevents* đến Gold Data để người đọc nhìn ra bài toán ngay lập tức.
- Gắn một citation nội bộ hoặc tài liệu thiết kế hệ thống cho phần mô tả workflow hiện tại của PREDICTCARE AI.

D. Mục tiêu nghiên cứu

Mục tiêu chính của bài báo cáo là đánh giá Ray như một giải pháp *all-in-one*, được định nghĩa là một nền tảng vận hành trên cùng một runtime thống nhất cho các tác vụ *distributed join*, ETL, huấn luyện mô hình và lập lịch tài nguyên. Cụ thể, nghiên cứu tập trung trả lời ba câu hỏi cốt lõi, tương ứng với ba tiêu chí đo đạc trong phần đánh giá. Thứ nhất là *tính thống nhất của kiến trúc*: liệu Ray có thể thay thế các pipeline phân mảnh truyền thống bằng một quy trình phân tán duy nhất cho bài toán xử lý đa bảng, từ đó giảm độ phức tạp vận hành và mã nguồn hay không. Thứ hai là *hiệu suất chuyển giao dữ liệu*: việc tích hợp Ray Data và Ray Train có giúp triệt tiêu các nút thắt cổ chai về disk I/O thông qua cơ chế bàn giao dữ liệu trong bộ nhớ hay không. Thứ ba là *khả năng xử lý dữ liệu quy mô lớn trên tài nguyên hạn chế*: liệu cơ chế chunking, distributed processing và streaming handoff có cho phép xử lý và “nén” khoảng 300 triệu bản ghi thô, tương đương gần

30GB dữ liệu, thành tập Gold Data đặc tả chỉ khoảng 12MB mà không gây lỗi tràn bộ nhớ RAM hệ thống trên các node CPU cloud hay không. Trên cơ sở đó, báo cáo xem Ray không chỉ là một framework tính toán, mà là *core engine* cho toàn bộ vòng đời dữ liệu và AI trong PREDICTCARE AI.

Phạm vi của nghiên cứu này tập trung vào việc xây dựng và tối ưu hóa *end-to-end pipeline* từ dữ liệu thô đến mô hình hoàn thiện. Các khía cạnh về triển khai dịch vụ mô hình hoặc suy luận trực tuyến không nằm trong phạm vi của bài báo cáo hiện tại.

Đóng góp chính của nghiên cứu gồm ba điểm. Thứ nhất, nghiên cứu thiết kế và triển khai một pipeline phân tích y tế toàn trình trên kiến trúc lai cloud-edge sử dụng Ray. Thứ hai, nghiên cứu đề xuất cơ chế nén dữ liệu chuỗi thời gian quy mô rất lớn thành tập dữ liệu đặc tả gọn nhẹ thông qua distributed join, windowing và aggregation. Thứ ba, nghiên cứu cung cấp nền tảng cho các đánh giá định lượng về hiệu quả sử dụng tài nguyên không đồng nhất giữa CPU cloud và GPU tiêu dùng RTX 3060 trong cùng một runtime thống nhất.

a) TO DO:

- Đối chiếu ba câu hỏi nghiên cứu với ba subsection trong phần evaluation để bảo đảm mỗi câu hỏi đều có số liệu trả lời trực tiếp.
- Điền số liệu thật cho luận điểm “30GB raw data → 12MB Gold Data” và gắn nó với chỉ số peak RAM của head node.
- Nếu cần, tách phần “Đóng góp chính” thành danh sách đánh số để làm nổi bật novelty của bài báo theo phong cách IEEE.
- Rà lại toàn bài để bỏ mọi câu chữ còn gợi ý rằng bài toán trung tâm là VRAM GPU thay vì data pipeline và System OOM ở phase ETL.

II. ARCHITECTURE OF MODERN DISTRIBUTED COMPUTING: THE RAY PARADIGM

A. Từ pipeline phân mảnh đến nền tảng tính toán thống nhất

Các hệ thống phân tích y tế hiện đại không chỉ cần xử lý dữ liệu lớn, mà còn cần di chuyển linh hoạt giữa các pha ingestion, distributed join, ETL, feature engineering, huấn luyện mô hình và suy luận. Kiến trúc phân tách công cụ truyền thống khiến từng pha được tối ưu cục bộ nhưng toàn pipeline lại chịu chi phí tích lũy lớn do serialization, disk I/O và độ trễ điều phối. Trong bối cảnh đó, Ray nổi lên như một mô hình mới, nơi xử lý dữ liệu, lập lịch tác vụ và huấn luyện phân tán có thể cùng hoạt động trong một runtime thống nhất. Cách tiếp cận này đặc biệt phù hợp với bài toán y tế, nơi pipeline không chỉ xử lý time-series mà còn phải kết nối nhiều bảng lâm sàng khác nhau trước khi downstream AI có thể bắt đầu.

Hình 1 minh họa trực quan khác biệt cốt lõi của bài báo: pipeline kiểu Spark truyền thống thường phải materialize dữ liệu trung gian xuống HDFS rồi nạp lại vào môi trường học máy riêng, trong khi Ray giữ các bước distributed join, aggregation và training trong một runtime thống nhất. Nhờ đó, trọng tâm của nghiên cứu chuyển từ việc tối ưu từng công cụ rời rạc sang tối ưu toàn bộ đường đi dữ liệu từ các bảng

MIMIC-IV thô đến Gold Data và mô hình XGBoost Survival Analysis.

a) *TO DO*:

- Bổ sung 1 đoạn văn ngắn so sánh kiến trúc “multi-engine” với “unified runtime” ở mức khái niệm, có trích dẫn paper hoặc docs của Ray.
- Chèn citation học thuật cho nhận định Ray là AI-native distributed framework, tránh để phần này chỉ dựa trên mô tả chung.
- Viết thêm 2–3 câu phân tích Hình 1, đặc biệt nhấn vào số lần materialization, số runtime và kiểu handoff dữ liệu.

B. *The Actor Model & Task-based Scheduling*

Một nền tảng lý thuyết quan trọng của Ray là mô hình tác vụ và actor. Thay vì gói toàn bộ workload vào các bước xử lý cứng kiểu stage-oriented như MapReduce, Ray cho phép biểu diễn công việc dưới dạng các *remote tasks* và *stateful actors*. Mô hình này tạo ra tính linh hoạt cao trong điều phối: các tác vụ join và ETL có thể được phân rã ở mức batch nhỏ, trong khi các actor dài sống có thể duy trì trạng thái cho các phiên huấn luyện hoặc bộ nạp dữ liệu. Với workload y tế, nơi một pipeline thường vừa có các bước xử lý song song mạnh, vừa có các bước cần giữ trạng thái tạm thời theo subject, admission hoặc cửa sổ thời gian, cơ chế này phù hợp hơn so với lối tổ chức cứng nhắc của các kiến trúc batch cổ điển.

Về thực tiễn, task-based scheduling còn cho phép Ray tận dụng cụm không đồng nhất tốt hơn. Các tác vụ join giữa *patients* và *admissions*, hoặc các phép aggregation trên *chartevents*, có thể được đẩy chủ yếu lên CPU node. Ngược lại, các tác vụ huấn luyện XGBoost Survival Analysis được lập lịch theo tài nguyên GPU. Nhờ đó, hệ thống không cần tách riêng một cluster ETL và một cluster AI; thay vào đó, scheduler của Ray đóng vai trò mặt phẳng điều phối chung cho toàn pipeline.

a) *TO DO*:

- Thêm trích dẫn paper Ray gốc cho actor model và task scheduling.
- Viết thêm 1 đoạn so sánh cụ thể với MapReduce hoặc Spark stage scheduling, tập trung vào tính mềm dẻo của tác vụ heterogeneous.
- Nếu có log hoặc dashboard, bổ sung ví dụ minh họa tác vụ CPU và GPU được schedule khác nhau trong cùng một job.
- Làm rõ actor nào trong pipeline thực tế của nhóm có thể là long-lived, actor nào chỉ là stateless batch task.

C. *In-Memory Plasma Store & Zero-Copy*

Điểm khác biệt trung tâm của Ray nằm ở cơ chế object store trong bộ nhớ, thường được nhắc đến qua Plasma Store. Thay vì yêu cầu mọi bước trong pipeline phải tuần tự hóa dữ liệu và truyền qua file hệ thống hoặc qua runtime nặng, Ray cho phép nhiều tiến trình trong cụm truy cập các object dùng chung với overhead thấp hơn. Đối với pipeline lâm sàng trong đề tài này, nơi intermediate table bao gồm bảng nhân sinh tồn, bảng đặc trưng time-series và Gold Data sau join cuối, khả

năng dùng chung object trong bộ nhớ giúp giảm sao chép dữ liệu và giảm áp lực ghi đọc đĩa.

Khái niệm *zero-copy* trong bối cảnh Ray đặc biệt quan trọng khi pipeline cần bàn giao dữ liệu từ khâu xử lý đa bảng sang khâu huấn luyện. Nếu dữ liệu trung gian luôn phải materialize thành file rồi được một framework khác đọc lại, hiệu năng toàn hệ thống sẽ bị giới hạn bởi tốc độ lưu trữ nhiều hơn là bởi năng lực tính toán. Ngược lại, khi Ray Data và Ray Train cùng nằm trong một mặt phẳng runtime, Gold Data có thể được chuyển giao dưới dạng object hoặc stream nội bộ, giúp giảm mạnh độ trễ và làm cho toàn pipeline gần với mô hình *in-memory distributed analytics* hơn là chuỗi batch job rời rạc.

a) *TO DO*:

- Tìm và trích dẫn nguồn mô tả Plasma Store hoặc object store architecture của Ray.
- Bổ sung 1 ví dụ rất cụ thể: nếu không có zero-copy thì bảng Gold Data sẽ phải được ghi và đọc lại như thế nào.
- Liên hệ trực tiếp với pipeline của nhóm bằng một mini-flow: “distributed join → Gold Data → streaming batches”.
- Nếu có thể, thêm sơ đồ mũi tên dữ liệu để làm rõ “no materialization back to HDFS”.

D. *Unified MLOps Ecosystem*

Một lý do quan trọng khiến Ray phù hợp với đề tài này là hệ sinh thái MLOps thống nhất. Ray Data đảm nhiệm đọc dữ liệu phân tán, ánh xạ hàm tiền xử lý, thực hiện distributed join, gom batch và điều phối dòng dữ liệu; Ray Train đảm nhiệm huấn luyện mô hình phân tán; còn các thành phần như Ray Tune hoặc Ray Serve mở ra khả năng tối ưu siêu tham số và triển khai mô hình trong các pha tiếp theo. Với góc nhìn hệ thống, ưu điểm lớn nhất không nằm ở từng module riêng lẻ, mà ở việc các module này chia sẻ cùng một triết lý runtime.

Trong đề tài này, sự liên mạch giữa Ray Data và Ray Train là trọng tâm. Sau khi ba bảng *patients*, *admissions* và *chartevents* được hợp nhất thành Gold Data, kết quả không cần quay lại HDFS như một bước trung gian bắt buộc. Thay vào đó, dòng dữ liệu đã xử lý có thể được đưa thẳng vào huấn luyện theo kiểu *streaming handoff*. Đây chính là điểm mà Ray mang lại lợi thế hệ thống vượt ra ngoài phạm vi của một engine ETL thuần túy.

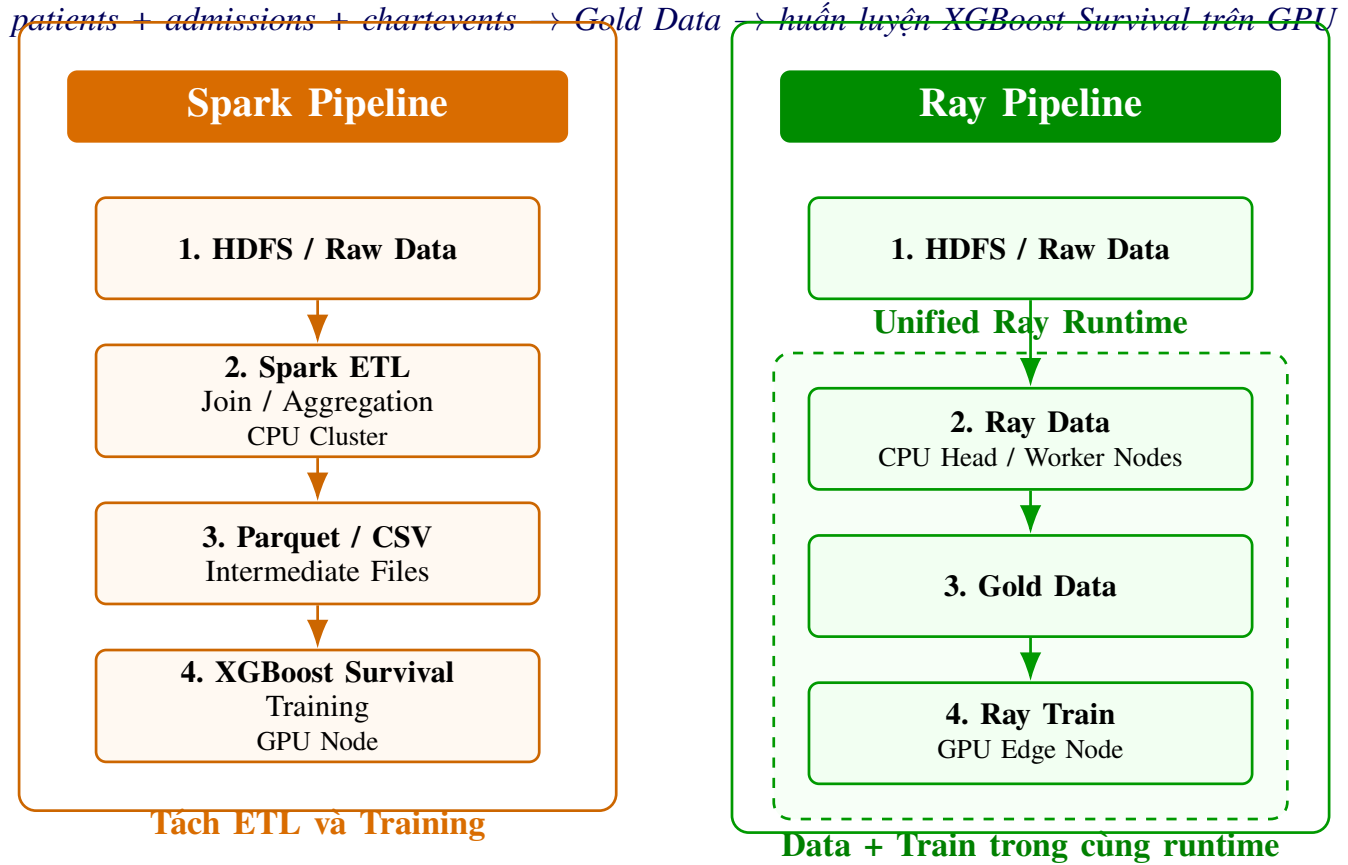
a) *TO DO*:

- Thêm 1 bảng mô tả vai trò của từng thành phần Ray trong bài: Ray Data, Ray Train, Ray Serve, Ray Tune.
- Viết rõ hơn phần nào của Ray đã dùng thật trong project, phần nào mới là hướng mở rộng tương lai.
- Bổ sung trích dẫn docs hoặc paper cho sự kết nối giữa Ray Data và Ray Train.
- Cần nhắc thêm một câu nêu giới hạn: bài này tập trung vào Data + Train, chưa benchmark Tune hoặc Serve.

E. *Kiến trúc lõi cần minh họa trong báo cáo*

Để làm rõ chiều sâu nghiên cứu, báo cáo nên chèn một hình kiến trúc Ray tại phần này, gồm ba lớp chính: (1) ứng dụng người dùng ở tầng trên cùng, (2) scheduler và runtime phân tán ở tầng giữa, và (3) object store cùng tài nguyên phần cứng

Kiến trúc pipeline: Spark vs Ray cho MIMIC-IV



Hình 1. Architecture-level comparison between a Spark-based pipeline and a Ray-based pipeline for the MIMIC-IV end-to-end analytics workflow.

CPU/GPU ở tầng dưới. Hình này giúp nhấn mạnh rằng Ray không chỉ là thư viện xử lý dữ liệu, mà là một kiến trúc hệ thống hoàn chỉnh cho AI-native distributed computing.

a) TO DO:

- Chuẩn bị một hình kiến trúc Ray sạch, ưu tiên tự vẽ lại theo phong cách đồng nhất của báo cáo thay vì chụp nguyên xi từ docs.
- Gắn nhãn rõ các thành phần cần liên hệ tới bài làm: head node, workers, object store, scheduler, CPU tasks, GPU training.
- Viết caption theo hướng giải thích ý nghĩa của hình, không chỉ mô tả tên thành phần.
- Đảm bảo figure này được viện dẫn trong thân bài bằng câu dẫn trước và câu phân tích sau hình.

III. SYSTEM DESIGN & METHODOLOGY

A. Heterogeneous Cloud-Edge Architecture

Hệ thống thực nghiệm được thiết kế theo mô hình cụm phân tán lai, trong đó tài nguyên được tách thành hai tầng với vai trò khác nhau nhưng được điều phối bởi cùng một Ray runtime. Tầng thứ nhất là *Cloud Head Node*, chịu trách nhiệm điều phối

job, lưu trữ metadata và đóng vai trò điểm truy cập tới lớp dữ liệu thô. Node này thiên về CPU và đồng thời kết nối với hệ thống lưu trữ phân tán HDFS. Tầng thứ hai là *Edge Compute Node*, sở hữu GPU NVIDIA RTX 3060 12GB và được dùng cho các pha huấn luyện cần tăng tốc phần cứng.

Hai tầng được kết nối qua mạng riêng ảo Tailscale, tạo thành một không gian điều phối thống nhất mặc dù phần cứng nằm ở các vị trí vận hành khác nhau. Cách mô tả này phản ánh đúng tinh thần *decoupled compute and storage*: lưu trữ và điều phối có thể đặt ở phía cloud hoặc hạ tầng máy chủ phổ thông, trong khi GPU tiêu dùng đặt ở edge vẫn được huy động như một tài nguyên huấn luyện hợp lệ của cả hệ thống. Trong báo cáo, kiến trúc này nên được minh họa bằng một sơ đồ topology mạng gồm HDFS, head node, worker CPU và edge GPU node.

a) TO DO:

- Điền thông số thật của từng node: CPU, RAM, ổ đĩa, GPU, hệ điều hành, phiên bản Ray, cách kết nối mạng.
- Vẽ topology mạng thể hiện rõ HDFS ở đâu, Ray head ở đâu, Ray worker/GPU ở đâu, Tailscale nối các node như thế nào.
- Viết rõ đường đi dữ liệu từ storage sang compute và từ

CPU node sang GPU node.

- Nếu chỉ có 2 máy, mô tả kiến trúc theo logic “cloud-edge” nhưng vẫn trung thực về số lượng node trong experimental setup.

B. Clinical Data Modeling: MIMIC-IV Multi-table Construction

Nguồn dữ liệu nghiên cứu được xây dựng từ ba bảng cốt lõi của MIMIC-IV là *patients*, *admissions* và *chartevents*. Bảng *patients* được dùng để xác định thông tin nền tảng của bệnh nhân và trạng thái tử vong. Bảng *admissions* cung cấp ngữ cảnh nhập viện, mốc thời gian bắt đầu theo dõi và khung *time-to-event*. Bảng *chartevents* chứa khoảng 300 triệu bản ghi sinh hiệu dạng time-series, phản ánh trạng thái lâm sàng động trong suốt quá trình chăm sóc.

Từ góc nhìn mô hình hóa, nghiên cứu tách bạch rõ ràng giữa *label* và *features*. Label sinh tồn được xây dựng từ quan hệ giữa *patients* và *admissions*, cụ thể là biến cố tử vong và thời gian từ lúc nhập viện đến biến cố hoặc kiểm duyệt. Ngược lại, features được trích xuất hoàn toàn từ *chartevents* thông qua các phép windowing và aggregation trên các cửa sổ lâm sàng sớm như 24 giờ hoặc 48 giờ đầu. Việc tách rõ hai lớp này giúp pipeline tránh rò rỉ nhãn và tạo nền tảng chặt chẽ cho survival analysis.

a) TO DO:

- Viết rõ schema tối thiểu của từng bảng: khóa join, cột thời gian, cột event, cột dùng cho feature extraction.
- Mô tả quy tắc tạo label bằng công thức hoặc pseudo-logic: event, censoring, time-to-event.
- Liệt kê danh sách sinh hiệu chính lấy từ *chartevents* và tiêu chí chọn chúng.
- Thêm một hình hoặc bảng “Label vs Features” để chứng minh không bị leakage giữa dữ liệu mục tiêu và dữ liệu đầu vào.

C. Analytical Model & Expected Output

Mô hình downstream được lựa chọn là XGBoost Survival Analysis với hàm mục tiêu `survival:cox`. Lựa chọn này phù hợp với bài toán dự báo tiên lượng lâm sàng vì mô hình không chỉ học quan hệ giữa đặc trưng và biến cố, mà còn tận dụng thông tin thời gian xảy ra biến cố trong dữ liệu. Với tập feature được xây dựng từ nhiều nguồn lâm sàng, mô hình kỳ vọng nắm bắt được ảnh hưởng tổng hợp của nhân khẩu học, bối cảnh nhập viện và biến thiên sinh hiệu sớm đến nguy cơ tử vong.

Đầu ra mà PREDICTCARE AI hướng tới không dừng ở một nhãn nhị phân. Thay vào đó, hệ thống cần cung cấp *Hazard Ratio* như một chỉ số rủi ro tương đối và *Survival Probability Curve* như một biểu diễn trực quan của xác suất sinh tồn theo thời gian. Hai đầu ra này có giá trị thực tế với bác sĩ vì cho phép đánh giá tiên lượng bệnh nhân theo hướng động, thay vì chỉ đưa ra một dự báo có hoặc không. Trong bối cảnh dữ liệu lớn và số lượng feature cao, GPU là yêu cầu gần như bắt buộc để rút ngắn thời gian hội tụ của XGBoost và làm cho huấn luyện trở nên thực tiễn.

a) TO DO:

- Ghi rõ thư viện hoặc implementation cụ thể đang dùng cho XGBoost survival:cox và các tham số chính dự kiến.
- Bổ sung một đoạn giải thích ngắn về ý nghĩa của hazard ratio và survival curve dưới góc nhìn lâm sàng.
- Thêm mô tả input/output của model ở dạng bảng: feature vector, label, prediction artifacts.
- Nếu có thể, đưa một hình minh họa survival curve mẫu để người đọc dễ hình dung đầu ra cuối cùng.

IV. END-TO-END PIPELINE IMPLEMENTATION

A. Distributed ETL via Ray Data

Pipeline toàn trình bắt đầu từ việc đọc dữ liệu thô từ HDFS vào Ray Data dưới dạng dataset phân tán. Bước đầu tiên là *label generation*, trong đó Ray thực hiện join giữa *patients* và *admissions* để tạo bảng nhãn sinh tồn. Tại bước này, trạng thái tử vong và mốc thời gian nhập viện được kết hợp để xác định biến cố, thời gian theo dõi và cấu trúc dữ liệu phù hợp cho objective `survival:cox`. Đây là lớp dữ liệu mục tiêu của toàn pipeline và được duy trì tách biệt với feature engineering nhằm bảo đảm tính đúng đắn thống kê.

Bước thứ hai là *windowing and aggregation* trên bảng *chartevents*. Thay vì cố gắng materialize toàn bộ 300 triệu dòng vào bộ nhớ, Ray Data đọc dữ liệu theo hướng streaming và sử dụng các phép biến đổi theo batch như `map_batches` để chuẩn hóa schema, lọc các chỉ số sinh hiệu quan trọng, đồng bộ thời gian theo admission, và tính các đặc trưng thống kê như *mean*, *max* và *min* trong các khung 24 giờ hoặc 48 giờ đầu. Nhờ đó, khối dữ liệu chuỗi thời gian khổng lồ được nén lại thành feature table gọn hơn nhưng vẫn giữ được tín hiệu lâm sàng chính.

Bước thứ ba là *final distributed join*. Bảng đặc trưng sinh hiệu sau aggregation được ghép nối với bảng nhãn đã xây dựng ở bước một để tạo ra Gold Data hoàn chỉnh. Đây là bước đặc biệt quan trọng trong đề tài vì nó thể hiện rõ năng lực của Ray đối với pipeline đa bảng: một phía là bảng đặc trưng thu gọn từ time-series quy mô cực lớn, phía còn lại là bảng nhãn được suy diễn từ dữ liệu nhân khẩu học và nhập viện. Kết quả cuối cùng là một dataset huấn luyện nhất quán, sẵn sàng chuyển thẳng sang mô hình survival.

a) TO DO:

- Viết pseudo-pipeline thật rõ theo 3 bước: label generation, aggregation, final distributed join.
- Bổ sung tên khóa join, điều kiện lọc thời gian, các itemid sinh hiệu và công thức aggregation đang dùng.
- Chèn một bảng “Input → Transformation → Output” cho từng bước ETL để người đọc thấy pipeline có cấu trúc.
- Chuẩn bị benchmark riêng cho bước join giữa bảng nhỏ và bảng lớn, vì đây là điểm hội đồng sẽ hỏi nhiều.

B. Seamless Handoff to GPU Training

Sau khi Gold Data được hình thành, hệ thống không đưa bảng này quay về HDFS như một pipeline hai pha cổ điển. Thay vào đó, Ray Data bàn giao trực tiếp kết quả đã được nén mạnh sang Ray Train trong cùng một runtime. Điểm mấu chốt ở đây không còn là “chia nhỏ để vừa VRAM”, mà là tận dụng

việc phase ETL đã cô đặc khối dữ liệu thô thành một ma trận đặc trưng dày đặc, gọn nhẹ và sẵn sàng cho huấn luyện tăng tốc bằng GPU.

Nhờ năng lực nén dữ liệu mạnh mẽ của Ray Data ở phase trước, khoảng 30GB dữ liệu thô từ *chartevents* và các bảng liên quan đã được tinh giản thành tập Gold Data đặc tả chỉ khoảng 12MB. Sự gọn nhẹ này cho phép Ray Train nạp toàn bộ ma trận đặc trưng vào thẳng bộ nhớ VRAM của GPU RTX 3060 trong một lần, theo đúng tinh thần *in-memory training*. Vì vậy, GPU không còn phải gánh vai trò “giải cứu dung lượng”, mà được sử dụng đúng thể mạnh là tăng tốc tính toán với 3584 nhân CUDA để đẩy nhanh tốc độ hội tụ của XGBoost Survival Analysis. Kết quả kỳ vọng là mô hình học được các cây quyết định phức tạp chỉ trong thời gian tính bằng giây sau khi phase data pipeline đã hoàn tất.

Code snippet suggestion: Chèn mã Python minh họa luồng join giữa *patients* và *admissions*, sau đó dùng `ray.data.read_parquet(...)` và `map_batches(preprocess)` để tạo feature table từ *chartevents*, cuối cùng stream Gold Data qua `iter_torch_batches()` hoặc `DataConfig` của Ray Train sang worker GPU.

Gợi ý trình bày: đặt đoạn mã minh họa này ngay sau phần mô tả cơ chế handoff giữa Ray Data và Ray Train.

a) TO DO:

- Thay khối “Code snippet suggestion” bằng code Python thật, đủ ngắn để lên paper nhưng đủ rõ luồng dữ liệu.
- Diễn số liệu thật về kích thước dữ liệu: raw input khoảng bao nhiêu GB, Gold Data sau nén còn bao nhiêu MB, số hàng và số cột đặc trưng cuối cùng.
- Ghi rõ cách Gold Data được nạp vào GPU và thời gian huấn luyện thực tế để chứng minh luận điểm “ultra-fast convergence on dense features”.
- Nếu có log hoặc screenshot từ phase training, trích phần thể hiện thời gian train ngắn và GPU utilization cao thay vì nhấn mạnh bài toán VRAM.
- Thêm một hình luồng dữ liệu “Raw Tables → Gold Data → In-memory Training on GPU” để minh họa đúng luận điểm mới.

V. EMPIRICAL EVALUATION

A. Performance & Execution Metrics

Đánh giá thực nghiệm tập trung vào hai thước đo chính: thời gian hoàn thành khâu Data Processing và thời gian hoàn thành huấn luyện mô hình sống còn. Ở pha Data Processing, báo cáo cần nhấn mạnh ba lớp chi phí: thời gian join *patients* với *admissions*, thời gian quét và aggregation trên *chartevents*, và thời gian final distributed join để tạo Gold Data. Điểm cần nhấn mạnh không chỉ là tốc độ, mà là việc hệ thống đã ép thành công khoảng 30GB dữ liệu thô thành tập dữ liệu huấn luyện đặc tả chỉ khoảng 12MB mà vẫn duy trì pipeline ổn định.

Một khía cạnh quan trọng khác là tính ổn định thực thi ở phase data pipeline. Trong pipeline cũ, join bảng lớn với bảng nhỏ và sau đó aggregate time-series thường phát sinh overhead bộ nhớ rất lớn ở RAM hệ thống, thậm chí dễ gây System OOM

trên head node nếu dữ liệu bị materialize không kiểm soát. Với Ray, distributed join và chunked processing được giữ trong cùng runtime, nhờ đó hệ thống có thể tạo Gold Data hoàn chỉnh trong khi mức tiêu thụ RAM đỉnh của head node vẫn nằm trong giới hạn chấp nhận được. Vì vậy, các chỉ số “peak RAM”, “compression ratio” và “hoàn thành thành công end-to-end” cần được xem là tiêu chí đánh giá quan trọng ngang với wall-clock time.

a) TO DO:

- Diễn số liệu thật cho từng pha: label generation, aggregation, final join, training; đặc biệt nhấn mạnh thời gian xử lý 30GB dữ liệu thô thành Gold Data 12MB.
- Chuẩn bị baseline để so sánh: PySpark cũ, pandas local hoặc pipeline cũ của nhóm, ưu tiên các trường hợp từng gặp chậm do join lớn hoặc vấp lỗi System OOM.
- Bổ sung một bảng tổng hợp runtime và một biểu đồ cột để nhìn nhanh chênh lệch giữa hệ cũ và Ray.
- Nếu có lỗi OOM cũ, mô tả ngắn tình huống đó như một baseline thất bại và đặt cạnh số liệu peak RAM hiện tại để tăng sức nặng cho kết quả.

B. Resource Utilization

Để chứng minh tính hợp lý của thiết kế cụm không đồng nhất, báo cáo nên đưa ảnh chụp từ Ray Dashboard vào phần này. Diễn giải mong muốn của biểu đồ là: ở phase 1, CPU của head node và các worker hoạt động ở mức cao do workload chủ yếu là ETL, distributed join và feature extraction; ở phase 2, mức sử dụng CPU giảm dần trong khi GPU trên edge node tăng mạnh khi bước huấn luyện bắt đầu. Mô hình chuyển pha tài nguyên này cho thấy pipeline đã tận dụng đúng vai trò của từng tầng phần cứng thay vì để GPU bị rỗi trong ETL hoặc để CPU tiếp tục là nút thắt trong training.

Ngoài CPU và GPU utilization, báo cáo cũng nên bình luận về network transfer giữa head node và edge node qua TailScale. Dù có thêm chi phí mạng, kiến trúc tách rời này vẫn có lợi vì cho phép tái sử dụng phần cứng tiêu dùng sẵn có, thay vì bắt buộc gom mọi tài nguyên vào một máy đơn nhiều chi phí.

Một chỉ số cần được nhấn mạnh thêm ở phần này là *peak RAM* của head node trong phase join và aggregation. Nếu chứng minh được hệ thống nén khoảng 30GB dữ liệu thô xuống 12MB Gold Data trong khi RAM đỉnh chỉ dao động quanh một ngưỡng X GB nhờ cơ chế chunking của Ray, thì giá trị kỹ thuật của data pipeline sẽ được thể hiện rõ ràng hơn nhiều so với việc chỉ báo cáo thời gian chạy.

a) TO DO:

- Chụp Ray Dashboard ở ít nhất 2 thời điểm: ETL/Join phase và Training phase.
- Chuẩn bị một hình hoặc bảng nhỏ tóm tắt CPU utilization, GPU utilization, peak RAM của head node và network activity.
- Viết phần giải thích hình theo logic “what the graph shows” thay vì chỉ nói “hình cho thấy tài nguyên được sử dụng tốt”.
- Bổ sung số liệu peak RAM thật của head node và gắn trực tiếp với luận điểm chunking/compression ở phase data pipeline.

C. Developer Experience

Một kết quả đáng chú ý của đề tài nằm ở trải nghiệm lập trình. Toàn bộ pipeline được mô tả bằng Python và vận hành trong cùng một hệ sinh thái Ray, thay vì phải phối hợp giữa nhiều ngôn ngữ hoặc nhiều tầng cấu trúc như Spark, YARN và các bước nạp dữ liệu thủ công cho framework ML khác. Điều này không chỉ rút ngắn thời gian phát triển, mà còn giúp việc debug, tái lập thí nghiệm và quan sát end-to-end trở nên dễ dàng hơn.

Từ góc độ nhóm phát triển PREDICTCARE AI, lợi ích quan trọng nhất là giảm chi phí nhận thức của hệ thống. Khi cùng một team có thể dùng một runtime để đi từ multi-table ingestion đến survival training, khả năng bảo trì và mở rộng về lâu dài sẽ tốt hơn đáng kể so với một kiến trúc ghép nối nhiều thành phần rời.

a) TO DO:

- Viết một bảng so sánh ngắn “Old stack vs Ray stack” về số công cụ, số bước, số lần materialize dữ liệu và độ khó vận hành.
- Ghi lại ví dụ thực tế từ lúc phát triển: số lượng script giảm đi, số bước debug giảm đi, hoặc chỗ nào Ray dashboard hỗ trợ debug tốt hơn.
- Thêm 1 nhận định trung thực về nhược điểm hoặc learning curve của Ray để phản đánh giá cân bằng hơn.
- Nếu có thể, đưa một sơ đồ nhỏ về developer workflow từ code đến execution để minh họa tính thống nhất của Python-only stack.

VI. KEY INSIGHTS, RECOMMENDATIONS & FUTURE WORK

A. Key Insights

Kết quả của nghiên cứu cho thấy giá trị lớn nhất của Ray không chỉ nằm ở việc tăng tốc một tác vụ phân tán riêng lẻ, mà ở khả năng hợp nhất toàn bộ vòng đời dữ liệu và AI trong cùng một nền tảng tính toán. Với bài toán MIMIC-IV, kiến trúc này giúp loại bỏ một phần lớn chi phí chuyển giao dữ liệu giữa distributed join, ETL và huấn luyện, đồng thời làm cho pipeline phù hợp hơn với các workload đa bảng có tính dị thể cao. Quan trọng hơn, việc tách rời lưu trữ và tính toán thông qua head node, HDFS và edge GPU node cho thấy Big Data không nhất thiết đòi hỏi hạ tầng đắt đỏ tập trung; ngược lại, phần cứng tiêu dùng sẵn có vẫn có thể được điều phối hiệu quả nếu có đúng runtime phân tán.

Một phát hiện quan trọng khác là kiến trúc cloud-edge qua TailScale giúp “bình dân hóa” Big Data và AI y tế. Bằng cách đặt tầng lưu trữ và điều phối ở cloud/head node, còn tầng tăng tốc ở edge node có GPU tiêu dùng, hệ thống vẫn duy trì được tính thống nhất end-to-end mà không cần đầu tư vào một máy chủ GPU đắt tiền. Với PREDICTCARE AI, đây là luận điểm công nghệ có ý nghĩa triển khai thực tiễn cao.

a) TO DO:

- Sau khi có kết quả thực nghiệm, rút ra 2–3 insight thật sắc bén thay cho các nhận định còn mang tính kỳ vọng.
- Viết lại insight theo cấu trúc “finding → evidence → implication” để phần kết luận mạnh hơn.
- Bảo đảm mỗi insight đều truy ngược được về số liệu hoặc hình ở phần evaluation.

B. Recommendations

Từ góc nhìn triển khai hệ thống, chúng tôi khuyến nghị nhóm PREDICTCARE AI xem Ray như *core engine* mới cho pipeline phân tích lâm sàng quy mô lớn. Ray Data nên được dùng để thay thế lớp ETL phân mảnh trước đây, đặc biệt trong các bước multi-table join giữa *patients*, *admissions* và *chartevents*. Trong khi đó, Ray Train nên đảm nhiệm pha huấn luyện XGBoost Survival Analysis trên GPU. Cách tiếp cận này mang lại ba lợi ích rõ ràng: giảm overhead do disk I/O trung gian, tăng khả năng tận dụng tài nguyên không đồng nhất, và đơn giản hóa stack phát triển khi toàn bộ pipeline được mô tả bằng Python.

Một khuyến nghị thực tiễn khác là ưu tiên duy trì kiến trúc *decoupled compute & storage*. Việc head node phụ trách điều phối và truy cập dữ liệu, còn edge node tập trung cho huấn luyện, là một cấu hình đặc biệt phù hợp với môi trường nghiên cứu hoặc startup có ngân sách phần cứng hạn chế nhưng vẫn muốn tiếp cận năng lực phân tích quy mô lớn.

a) TO DO:

- Viết recommendation theo mức độ ưu tiên: ngắn hạn, trung hạn, dài hạn để người đọc thấy khả năng triển khai thực tế.
- Gắn mỗi recommendation với điều kiện áp dụng, ví dụ khi dữ liệu tăng quy mô bao nhiêu hoặc khi team cần huấn luyện trên GPU consumer.
- Bổ sung một câu nêu rõ khi nào Ray chưa phải lựa chọn tốt nhất để recommendation cân bằng và đáng tin hơn.

C. Future Work

Hướng phát triển tiếp theo của đề tài là mở rộng Ray từ tầng xử lý và huấn luyện sang tầng phục vụ mô hình. Cụ thể, Ray Serve có thể được dùng để triển khai mô hình XGBoost Survival Analysis thành API cho PREDICTCARE AI, cho phép dashboard hoặc ứng dụng lâm sàng truy cập trực tiếp hazard ratio và survival probability curve của từng bệnh nhân. Xa hơn, hệ thống có thể kết hợp với các Large Language Models để xây dựng giao diện hỏi đáp bằng ngôn ngữ tự nhiên, giúp bác sĩ tương tác với kết quả đường cong sinh tồn hoặc nhận giải thích lâm sàng một cách trực quan hơn.

Ngoài ra, nghiên cứu tương lai nên đánh giá sâu hơn trên các cấu hình nhiều GPU hoặc nhiều edge node, cũng như bổ sung các thước đo định lượng về network throughput, memory footprint và chi phí vận hành. Những kết quả này sẽ giúp hoàn thiện luận điểm rằng Ray không chỉ phù hợp cho proof-of-concept học thuật, mà còn có tiềm năng trở thành nền tảng sản xuất thực sự cho phân tích y tế quy mô lớn.

a) TO DO:

- Tách hướng tương lai thành 3 nhánh rõ ràng: serving, scale-out multi-GPU/multi-node, và giao diện tương tác với LLM.
- Viết cụ thể từng nhánh cần làm gì đầu tiên, ví dụ: expose API nào qua Ray Serve, cần artifact nào để dựng survival curve online.
- Thêm một câu về đánh giá đạo đức hoặc kiểm định lâm sàng nếu hệ thống được đưa gần hơn tới môi trường thực tế.

- Liên hệ future work với Topic 3 của giảng viên một cách tự nhiên, tránh làm đoạn này trở nên rời rạc với phần thân bài.

TÀI LIỆU

- [1] PREDICTCARE AI Capstone Project Internal Design Notes, unpublished internal document, 2026.
- [2] A. Johnson, T. Pollard, L. Shen, et al., “MIMIC-IV, a freely accessible electronic health record dataset,” *Scientific Data*, vol. 10, no. 1, 2023.
- [3] P. Moritz, R. Nishihara, S. Wang, et al., “Ray: A Distributed Framework for Emerging AI Applications,” in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [5] H. Ishwaran, U. B. Kogalur, E. H. Blackstone, and M. S. Lauer, “Random survival forests,” *The Annals of Applied Statistics*, vol. 2, no. 3, pp. 841–860, 2008.
- [6] The Ray Team, “Ray Documentation: Ray Data, Ray Train, and Ray Serve,” technical documentation, 2026.